



Instrucciones de Competencia Final — Proyecto ARTEMIS

Curso Modelos Avanzados de Procesamiento de Lenguaje Natural

1 Número de grupo

El número de grupo para esta competencia se encuentra en la lección de Entregables de la Competencia.

2 Introducción

Bienvenido a la competencia final del curso Modelos Avanzados de Procesamiento de Lenguaje Natural. En este desafío, los participantes deberán construir un sistema de Retrieval-Augmented Generation (RAG) con *Tool Calling*. El objetivo es diseñar un sistema capaz de, dado un *query* en lenguaje natural emitido por un operador del centro de control de la agencia espacial ficticia MASA, predecir la *tool call* correcta en formato canónico, utilizando documentación técnica recuperada como contexto.

3 Resultados de Aprendizaje

Con esta actividad se busca que el estudiante pueda poner en práctica el desarrollo de una solución completa de *NLP* para un problema de *retrieval* y generación condicionada. Tras realizar esta actividad, se espera que el estudiante esté en capacidad de:

1. Diseñar e implementar sistemas de *retrieval* densos basados en *encoders* pre-entrenados.
2. Realizar *fine-tuning* de modelos de lenguaje de tipo *decoder* para tareas de generación estructurada.
3. Construir *pipelines* de RAG que combinan *retrieval* (FAISS) con generación condicionada por contexto recuperado.
4. Aplicar técnicas de *data augmentation*, *hybrid search* y *reranking* para mejorar el desempeño.
5. Diseñar estrategias de *prompting*, post-procesamiento y normalización de *output* para tareas con formato estricto.

4 Descripción de la Competencia

Usted y su grupo deberán participar en una competencia en Kaggle diseñada para este curso. La competencia consiste en predecir la *tool call* correcta para cada *query* de un operador del centro de control de la estación espacial Kuntur Station, operada por la agencia ficticia MASA (Misión Aeroespacial Sudamericana Avanzada). Cada *query* puede requerir información presente en los documentos técnicos del centro de control para determinar parámetros como severidad o protocolos de emergencia. El reto es construir un sistema que, dada una consulta en inglés, recupere los documentos relevantes y genere

la *tool call* canónica correspondiente. La competencia es exclusiva para estudiantes de la Maestría en Inteligencia Artificial de la Universidad de los Andes inscritos en el primer ciclo de 2026-10 en la materia de Modelos Avanzados de Procesamiento de Lenguaje Natural.

4.1 Contexto de la misión

Estamos en 2026. Hace apenas unos días, la misión Artemis II de la NASA sobrevoló la Luna con cuatro astronautas a bordo, marcando el regreso de la humanidad a la órbita lunar después de más de 50 años. El programa Artemis avanza hacia el alunizaje tripulado y, eventualmente, hacia la presencia humana permanente en el espacio profundo.

En este contexto, la Misión Aeroespacial Sudamericana Avanzada (MASA) — la primera agencia espacial latinoamericana — ha puesto en órbita la estación Kuntur Station. A bordo viajan seis tripulantes en una misión de larga duración: el Comandante Santiago Reyes, la Piloto Ana Valdivia, los Especialistas Kai Nakamura y Fatima Al-Hassan, el Ingeniero Pavel Kozlov y la Oficial Médico Lucía Mendoza. Están solos a 400 km sobre la Tierra, y cualquier decisión equivocada puede poner en riesgo sus vidas.

La estación cuenta con 6 módulos, cada uno bautizado con fauna emblemática de Sudamérica:

- **Cóndor** — Comando y control (el cerebro de la estación).
- **Quetzal** — Laboratorio científico (4 tripulantes, 12 estaciones de trabajo).
- **Jaguar** — Soporte vital y sistemas críticos (redundancia Clase A).
- **Colibrí** — Comunicaciones y navegación (antenas de largo alcance).
- **Vicuña** — Almacenamiento, carga y puerto de acoplamiento.
- **Tucán** — Habitacional (6 cabinas individuales para la tripulación).

Desde el centro de control en tierra, los operadores envían consultas al sistema para monitorear la estación, ejecutar protocolos de emergencia, coordinar a la tripulación y mantener los sistemas funcionando. Su tarea es construir el agente inteligente que interprete esas consultas y ejecute la acción correcta. Un operador podría preguntar sobre lecturas anómalas de oxígeno en Jaguar, solicitar activar un protocolo de emergencia por radiación en Quetzal, o pedir la ubicación actual de un tripulante. Su sistema debe entender la consulta, recuperar la documentación técnica relevante y generar la *tool call* precisa que resuelve la situación. La tripulación de Kuntur Station cuenta con ustedes.

4.2 Las 10 Tools Disponibles

El sistema cuenta con 10 *tools* disponibles, todas con parámetros tipo `enum` (sin texto libre). A continuación se listan junto con sus parámetros y valores válidos:

Tool	Parámetro	Tipo	Valores válidos
get_telemetry	module metric timeframe_hours	str str int	condor, quetzal, jaguar, colibri, vicuna, tucan temperature, pressure, oxygen, radiation, humidity, power 1, 6, 12, 24
get_crew_status	module info	str str	condor, quetzal, jaguar, colibri, vicuna, tucan health, location, current_activity, schedule
get_module_status	module system	str str	condor, quetzal, jaguar, colibri, vicuna, tucan life_support, power, thermal, structural, communications
send_alert	module severity reason	str str str	condor, quetzal, jaguar, colibri, vicuna, tucan low, medium, high, critical abnormal_temperature, pressure_drop, oxygen_leak, radiation_spike, system_failure, power_fluctuation, communication_loss, structural_damage
send_message	recipient priority	str str	commander, pilot, specialist_1, specialist_2, engineer, medical_officer, all_crew low, medium, high, urgent
schedule_maintenance	module task priority	str str str	condor, quetzal, jaguar, colibri, vicuna, tucan sensor_repair, filter_replacement, system_calibration, hull_inspection, power_cell_swap, software_update routine, urgent
activate_protocol	protocol_id scope	str str	MASA-SEC-001 ... MASA-SEC-020 module_only, station_wide
control_system	module system action	str str str	condor, quetzal, jaguar, colibri, vicuna, tucan ventilation, heating, lighting, cooling, filtration activate, deactivate, increase, decrease
calculate_trajectory	maneuver urgency	str str	orbit_adjustment, docking, debris_avoidance, reentry, station_keeping planned, immediate
request_supply	category urgency	str str	medical, food, equipment, fuel, spare_parts, scientific routine, expedited, emergency
no_action	—	—	Sin parámetros (consultas informativas / históricas)

Table 1: Definición completa de las 10 tools disponibles. **El orden en que aparecen los parámetros en esta tabla es el orden obligatorio en la tool call.**

Importante: para cada *tool*, los parámetros deben aparecer en el mismo orden de arriba a abajo en el que se listan en la Tabla 1. Por ejemplo, `send_alert` define primero `module`, luego `severity`, luego `reason`; cualquier otro orden se evalúa como incorrecto. El archivo `tools_definition.json` contiene este mismo orden de forma procesable. A continuación se muestra un ejemplo válido por cada *tool*:

```
get_telemetry(module='tucan',metric='radiation',timeframe_hours=24)
get_crew_status(module='tucan',info='health')
get_module_status(module='tucan',system='life_support')
send_alert(module='vicuna',severity='critical',reason='radiation_spike')
send_message(recipient='all_crew',priority='low')
schedule_maintenance(module='quetzal',task='sensor_repair',priority='routine')
activate_protocol(protocol_id='MASA-SEC-008',scope='module_only')
control_system(module='jaguar',system='heating',action='deactivate')
calculate_trajectory(maneuver='reentry',urgency='planned')
request_supply(category='scientific',urgency='emergency')
no_action
```

Nota importante: Los nombres de los módulos en los documentos técnicos conservan su grafía original en español (Cóndor, Colibrí, Vicuña, Tucán), mientras que los valores válidos en las tool calls usan su forma ASCII sin acentos (condor, colibri, vicuna, tucan). Tenga en cuenta que estos nombres propios provienen de lenguas indígenas y del español, y no forman parte del vocabulario estándar de modelos entrenados en inglés.

4.3 Requisitos técnicos

Recuerde que su modelo no tendrá validez si no cumple los requisitos que se mencionan a continuación. Por lo tanto, obtendrá una calificación de 0 en la actividad correspondiente al proyecto final. Los requisitos son los siguientes:

- El *decoder* obligatorio es meta-llama/Llama-3.2-1B-Instruct (link). No se permite usar otro *decoder*.
- El *encoder* obligatorio es BAAI/bge-small-en-v1.5 (link). No se permite usar otro *encoder*.

- Está prohibido el uso de modelos pagos (OpenAI, Claude, Cohere, Gemini APIs u otros servicios de pago).
- Está prohibido usar datos externos no entregados por el equipo docente.
- Está prohibido utilizar los datos de prueba para entrenar (*data leakage*).
- Está prohibido reemplazar el LLM por otra red neuronal de clasificación.
- Es obligatorio realizar *fine-tuning* del *decoder*; no se acepta solo *prompt engineering*.
- El entrenamiento debe realizarse en PyTorch.
- Los resultados deben ser replicables a partir de los entregables en la plataforma de Coursera, partiendo del conjunto de entrenamiento original entregado.

5 Envío de resultados por la plataforma Kaggle

Para enviar los resultados, genere un archivo CSV con las predicciones para todas las 766 consultas de `test.csv` y súbalo en la plataforma Kaggle. La plataforma calculará automáticamente su score según la metodología estándar de Kaggle (score público visible durante la competencia y score privado revelado al cierre).

La métrica de evaluación es *Categorization Accuracy (exact string match)*: la predicción debe coincidir carácter por carácter con la *tool call* canónica correcta. No se otorga crédito parcial.

Nombre del equipo en Kaggle: cada grupo debe registrarse con el nombre `Grupo_N`, donde N es el número de su grupo asignado en Coursera. Ejemplos: `Grupo_1`, `Grupo_2`, `Grupo_10`. Es obligatorio usar el mismo número de grupo que tienen en la plataforma de Coursera, ya que este será el identificador para asociar los resultados de Kaggle con la nota del grupo.

6 Entregables en Coursera

En la semana 8 del curso se habilitará una actividad en donde usted y su grupo deben entregar los siguientes artefactos correspondientes únicamente al modelo final con el que obtuvieron los mejores resultados en la tabla de líderes pública de la competencia de Kaggle. Todos los entregables deben empaquetarse en un único archivo `.zip` nombrado `entrega_grupo_XX.zip` y subirse a una carpeta compartida de Google Drive cuyo enlace se proporcionará en Coursera.

El `.zip` debe contener:

1. **Notebook principal** con el *pipeline* completo, organizado en secciones claramente diferenciadas: (1) carga y exploración de datos, (2) entrenamiento del o los modelos, (3) construcción del índice de *retrieval*, (4) inferencia, y (5) generación del archivo de respuesta. Toda *data augmentation* o generación adicional de texto debe quedar documentada en el *notebook*. Si el grupo utiliza *scripts* de Python externos que se invocan desde el *notebook*, está permitido siempre que todos los *scripts* estén incluidos en el `.zip`.

El *notebook* debe seguir una estructura clara con secciones marcadas. Ejemplo mínimo:

```
# 1. Carga y Exploracion de Datos
- Carga de train.csv, consultas_centro_control.json, documentos MASA
- Analisis exploratorio: distribucion de tools, duplicados, estadisticas

# 2. Preprocesamiento
- Limpieza de datos (duplicados, ruido)
- Chunking de documentos
- Generacion de datos adicionales (si aplica, justificar modelo usado)

# 3. Entrenamiento
- Fine-tuning del encoder (si aplica)
- Construccion del indice FAISS
- Fine-tuning del decoder (obligatorio)

# 4. Inferencia
- Carga de modelos entrenados
```

```
- Pipeline completo: query -> retrieval -> generacion -> post-procesamiento

# 5. Generacion del Archivo de Respuesta
- Generacion de submission.csv para Kaggle
```

2. **Mejor modelo del decoder** (Llama-3.2-1B-Instruct *fine-tuneado*): el *checkpoint* completo del mejor modelo obtenido durante el entrenamiento (directorio generado por `model.save_pretrained()`).
3. **Mejor modelo del encoder** (si aplica): si realizaron *fine-tuning* del *encoder* (`bge-small-en-v1.5`), incluir el *checkpoint* completo del mejor modelo.
4. **Índice de retrieval** en formato JSON con la siguiente estructura por cada *chunk* indexado:

```
[
  {
    "doc_id": "MASA-DOC-007",
    "chunk_id": 0,
    "text": "...contenido del chunk...",
    "vector": [0.123, -0.456, ...]
  },
  {
    "doc_id": "MASA-DOC-007",
    "chunk_id": 1,
    "text": "...siguiente chunk del mismo documento...",
    "vector": [0.789, -0.012, ...]
  }
]
```

Si un documento se divide en múltiples *chunks*, el `doc_id` se repite y cada `chunk_id` es único, tanto dentro de ese documento como de forma global.

Estructura esperada del `.zip`:

```
entrega_grupo_XX.zip/
  pipeline.ipynb # Notebook principal
  scripts/ # Scripts externos (si aplica)
    chunker.py
    finetune_decoder.py
    utils.py
  prompts/ # Prompts utilizados (si aplica)
    system_prompt.yaml
  models/
    decoder/ # Checkpoint del decoder fine-tuneado
      adapter_config.json
      adapter_model.safetensors
      tokenizer.json
      ...
    encoder/ # Checkpoint del encoder (si aplica)
      config.json
      model.safetensors
      tokenizer.json
      ...
  retrieval_index.json # Índice de retrieval con chunks y vectores
  requirements.txt # Dependencias
```

Entrega en Coursera: en la actividad habilitada en Coursera, suba únicamente el enlace de Google Drive donde se encuentra el archivo

entrega_grupo_XX.zip. Asegúrese de que el enlace tenga permisos de lectura para cualquier persona con el enlace.

6.1 Sobre el uso de modelos adicionales para data augmentation

Si el grupo decide generar datos sintéticos adicionales para mejorar el entrenamiento, puede utilizar modelos de lenguaje *open-source* disponibles en Hugging Face (descargados localmente) o NVIDIA BUILD, en este último, también se puede través del *endpoint* gratuito de NVIDIA Build (build.nvidia.com/models). En ambos casos, el grupo debe justificar la elección del modelo (capacidades, licencia, motivación) dentro del mismo *notebook* entregado. No se aceptan modelos pagos ni datos externos no generados por el grupo.

7 Rúbrica de Evaluación

La nota del proyecto se compone de los siguientes criterios:

Table 2: Composición de la nota del proyecto final

Item	Porcentaje de la nota
Participación en la competencia	20%
Superar la línea base (ver en Kaggle)	40%
Percentil obtenido en la competencia	40%

Advertencia sobre entregables — penalizaciones:

- Si se incumple cualquier regla de la sección 4.3 la nota total de la competencia será **cero**.
- El `.zip` entregado debe respetar exactamente la estructura de carpetas que se describe en la sección 5. Entregas que no sigan esta estructura recibirán una penalización de **1.0 unidad** sobre la nota final del proyecto.
- El *notebook* principal debe ser autocontenido: al ejecutarse de inicio a fin (junto con los *scripts* incluidos en el `.zip`), debe reproducir todo el *pipeline* — desde la carga de datos hasta la generación del archivo de *submission*. Un *notebook* que no sea ejecutable de forma autónoma o que dependa de rutas absolutas externas recibirá una penalización de **1.0 unidad** sobre la nota final. Un entregable que no se pueda ejecutar ni verificar no puede evaluarse.
- El archivo debe nombrarse `entrega_grupo_XX.zip` (donde `XX` es el número de grupo en Coursera). El número del equipo en Kaggle debe ser el mismo número del equipo en Coursera, si cualquiera de estos dos criterios no se cumplen (**Grupo N**), las entregas con un nombre de archivo incorrecto o número de equipo distinto en Kaggle recibirán una penalización de **0.3 unidades** sobre la nota final.

7.1 Participación en la competencia

Para que sea considerada su participación en la competencia, debe realizar al menos 5 envíos **diferentes** en la plataforma de Kaggle. La idea es evidenciar que el grupo trató de mejorar su enfoque inicial. Tiene dos semanas para cumplir con ese número mínimo de envíos. No hay límite máximo de envíos totales, pero se pueden hacer hasta 5 envíos diarios por equipo.

7.2 Superar la línea base

El equipo del curso preparó una línea base que debe superar. Esta línea base la encuentra publicada en el *leaderboard* de la competencia en Kaggle. Si no se supera la línea base, esta porción de la nota es 0.0.

7.3 Percentil obtenido en la competencia

El 40% restante de la nota final se obtiene de acuerdo al desempeño del modelo desarrollado con su grupo en la competencia de Kaggle. Al cerrar la competencia se generan unos puntajes finales, y su nota en este rubro dependerá del desempeño obtenido. El porcentaje obtenido de este rubro se calcula de la siguiente manera:

Desempeño obtenido	Porcentaje del rubro
< percentil 25	0%
≥ percentil 25 y < percentil 50	50%
≥ percentil 50 y < percentil 75	75%
≥ percentil 75	100%

Table 3: Distribución del porcentaje del rubro según desempeño obtenido

7.4 Fechas Importantes

- Fecha de apertura de la competencia: **10/05/2026 12:00 AM UTC-5**
- Fecha de finalización: **24/05/2026 11:59 PM UTC-5**

8 Descripción de los Datos

Se proporcionan los siguientes archivos:

- **train.csv**: contiene 2718 ejemplos de entrenamiento correspondientes a consultas de operadores del centro de control. Contiene datos duplicados y ruidosos — parte del reto es limpiarlos adecuadamente. Se recomienda particionar este archivo en subconjuntos de entrenamiento y validación según la estrategia del grupo. Columnas:
 - **id**: identificador único de la consulta (formato Q-XXXXX).
 - **query**: consulta en lenguaje natural emitida por un operador.
 - **tool_call**: *tool call* correcta en formato canónico (variable objetivo).
- **test.csv**: archivo de evaluación que contiene 766 consultas sin respuesta. No hay superposición con **train.csv**. Columnas:
 - **id**: identificador único de la consulta (formato Q-XXXXX).
 - **query**: consulta en lenguaje natural a la cual se debe predecir la *tool call*.
- **consultas_centro_control.json**: archivo con 810 pares (**query**, **doc_id**) que registran consultas históricas del centro de control junto con el documento técnico que las responde. Algunas entradas incluyen además un campo **hard_negative_doc_id** que indica un documento que podría confundirse con el correcto. Cada entrada tiene la forma:

```
{ "query": "Which protocol governs radiation lockdown procedures?",  
  "doc_id": "MASA-DOC-009",  
  "hard_negative_doc_id": "MASA-DOC-012" }
```

- **base_conocimiento/**: carpeta que contiene los 54 documentos técnicos de MASA (manuales de módulos, protocolos de seguridad, procedimientos operacionales y guías de sistemas) que conforman el *corpus* de *retrieval*. Cada documento se entrega como archivo Markdown bajo una carpeta nombrada con su identificador (MASA-DOC-XXX/doc.md).
- **tools_definition.json**: definición completa de las 10 *tools* disponibles en formato JSON procesable, incluyendo todos los parámetros válidos por enumeración. Corresponde a la tabla presentada en la Sección 3.2.
- **sample_submission.csv**: ejemplo del formato de entrega esperado en Kaggle para **test.csv**. Contiene una fila por cada **id** del archivo **test.csv** con un valor *placeholder* en la columna **tool_call**, que el estudiante debe reemplazar con la predicción de su modelo.

8.1 Formato de respuesta canónico

Las respuestas deben seguir un formato estricto de *tool call*. La métrica de evaluación es *exact match* del *string* completo; no se otorga crédito parcial. En un entorno operativo real, cualquier discrepancia impediría la ejecución de la herramienta y pondría en riesgo la misión. Se permite post-procesamiento del *output* (regex, normalización de formato). Las reglas del formato canónico son:

1. Orden de parámetros: los parámetros deben aparecer en el mismo orden definido en **tools_definition.json** y en la Tabla 1 del presente documento. Una *tool call* con parámetros correctos pero en orden incorrecto se evalúa como incorrecta (0 puntos).
2. Sin espacios después de comas ni antes/después del signo =.
3. Comillas simples para valores tipo *string*.

4. Todo en minúsculas, excepto `protocol_id` que va en MAYÚSCULAS (e.g., `MASA-SEC-001`).
5. Números sin comillas (e.g., `timeframe_hours=1`, no `timeframe_hours='1'`).

Ejemplos de tool calls válidos:

```
send_alert(module='quetzal',severity='critical',reason='abnormal_temperature')
get_telemetry(module='condor',metric='pressure',timeframe_hours=1)
calculate_trajectory(manuever='docking',urgency='planned')
no_action
```

Ejemplo de tool call INVÁLIDO (orden incorrecto):

```
send_alert(severity='critical',module='quetzal',reason='abnormal_temperature')
```

Aunque los valores sean correctos, el orden de los parámetros no coincide con la definición de la *tool*, por lo tanto la predicción se evalúa como incorrecta.

8.2 Archivo de respuesta

El archivo de respuesta debe contener el siguiente formato:

```
id,tool_call
T-00001,activate_protocol(protocol_id='MASA-SEC-009',scope='module_only')
T-00002,no_action
T-00003,get_telemetry(module='jaguar',metric='oxygen',timeframe_hours=12)
...
```

Cada *id* en el conjunto de prueba debe tener un valor en la columna `tool_call`, indicando la *tool call* predicha.

8.3 Recomendaciones

Se recomienda encarecidamente consultar el estado del arte en áreas como *Retrieval-Augmented Generation*, *tool calling*, *fine-tuning* de modelos de lenguaje pequeños y técnicas de *dense retrieval*. Revisar artículos científicos recientes y documentación técnica de las librerías utilizadas puede marcar la diferencia entre un sistema que apenas supera la línea base y uno que alcanza un desempeño competitivo. Algunos ejes de exploración relevantes incluyen: estrategias de *chunking*, funciones de pérdida para *retrieval* y *data augmentation* para generación condicionada

9 Enlaces de la Competencia

- Enlace de Invitación: [Kaggle Invitation](#)